REVA UNIVERSITY
Bengaluru, India

School of Computer Science and Engineering

Course Code:B25CSS711	**Capstone project Report**	**Academic Year:** 4th
		Semester: 7th

Project Group Members Details

Group No:	**Group Name:** Team Dynamites

Sl. No	SRN	Full Name	Sec	Mobile No.	Email_ID	Sign
1	R23EF022	Aneesh J Prakash	A	7899578024	Aneeshj09012006@gmail.com	
2	R23EF059	D Naga Sumukh	A	9448794069	nagasumukh01@gmail.com	

Task Distribution

Project Leader	Aneesh J Prakash
Documenter Lead:	D Naga Sumukh

Project Details

Project Title:	**AI-Powered Fault Prediction in 5G Testbed**	
Type of Project	Capstone project (AI/ML – Telecom Network Research)	
Guide Details:	**Name: Dr. Afifa Salsabil Fathima A** **Designation: Assistant Professor**	**Mobile No: 89700 68031**
Place of Project Work:	REVA University, Bangalore	
Remarks by Guide:		**Guide Signature** **Date:**

Contents

Abstract	
Introduction	Pg. 1
Problem Definition & Objectives	Pg. 2
Design & Methodology	Pg. 3-4
Implementation & Functionality	Pg. 5-6
Result & Discussion	Pg. 7-8
Conclusion	Pg. 9

Abstract

Conventional 5G and O-RAN network operations depend on static threshold alarms that fire only after quality of service has already degraded, leaving operators no time to act. This project presents "AI-Powered Fault Prediction in 5G Testbed", a proactive platform that learns from multi-cell 5G telemetry to forecast service disruptions 15 to 22.5 minutes before an SLA breach. A telemetry simulator injects eight fault scenarios, including radio degradation, cell congestion, packet loss, latency, throughput loss, compute saturation, backhaul interruption and core availability degradation. A Random Forest classifier, with Logistic Regression as a baseline, predicts impending faults, while an unsupervised Isolation Forest detects zero-day anomalies. Predictions are converted into composite risk scores (LOW to CRITICAL) and managed through an incident lifecycle with operator audit trails and mitigation advisories. The system is built with FastAPI, a React operations centre, and Prometheus/Grafana monitoring, and can be deployed locally, with Docker Compose, or on Render and Vercel. Evaluated on a strictly chronological held-out set of 3,150 samples, the Random Forest achieved 99.32% precision, 100% recall and a 99.66% F1-score with no missed faults, compared with 60% recall for static thresholds. The results suggest that machine-learning-based early warning can substantially improve fault detection in 5G networks, although validation on real operator data is needed.

Keywords: *5G, O-RAN, fault prediction, anomaly detection, Random Forest, Isolation Forest, network operations, FastAPI, React.*

1. Introduction

1.1 Background

Fifth-generation (5G) and Open RAN (O-RAN) networks carry latency-critical and bandwidth-intensive services such as ultra-reliable low-latency communication (URLLC), enhanced mobile broadband and massive machine-type communication. Keeping these services within their service-level agreements (SLAs) requires operators to detect degradation early. In practice, most network operations centres still depend on reactive, static threshold alarms: an alert is raised only after a key performance indicator (KPI) has already crossed its limit and the quality of service has collapsed.

This mini project, "AI-Powered Fault Prediction in 5G Testbed", was developed by Team Dynamites. It is an autonomous, proactive platform that learns from multi-cell 5G telemetry and forecasts service disruptions 15 to 22.5 minutes before an SLA breach, giving operators time to act rather than react.

1.2 Scope of the Project

The platform covers the complete workflow from telemetry generation to operator action:

- a 5G telemetry simulator that injects eight realistic fault scenarios into multi-site, multi-cell data;
- supervised fault prediction using Random Forest (primary) and Logistic Regression (baseline) models;
- unsupervised zero-day anomaly detection using an Isolation Forest;
- composite risk scoring (LOW, MEDIUM, HIGH, CRITICAL) and a managed incident lifecycle with operator audit trails;
- a FastAPI backend, a React-based network operations centre (NOC) dashboard, and Prometheus/Grafana monitoring with a mock ONAP VES event dispatcher.

1.3 Organisation of the Report

Section 2 defines the problem and objectives. Section 3 describes the system design and methodology. Section 4 explains the implementation and functionality, including deployment. Section 5 presents and discusses the evaluation results, and Section 6 concludes the report.

2. Problem Definition & Objectives

2.1 Problem Definition

Traditional 5G operations rely on static 3GPP-style threshold alarms. These alarms have three weaknesses that motivate this work:

- **Reactive by design.** An alarm fires only after the KPI limit is breached, so the lead time available to the operator is zero minutes.
- **Poor sensitivity.** Single-metric thresholds miss faults that develop gradually or across several correlated KPIs; on the held-out test set the static-threshold baseline missed 40% of faults.
- **No guidance.** A threshold alarm reports that something is wrong but not why, nor what mitigation to apply.

The problem addressed by this project is therefore: given historical and streaming multi-cell 5G telemetry, predict impending service-affecting faults early and accurately enough to allow corrective action before the SLA is breached, while also surfacing previously unseen (zero-day) anomalies and recommending mitigation.

2.2 Objectives

- Forecast network service disruptions 15 to 22.5 minutes before SLA breach.
- Detect zero-day multivariate anomalies with an unsupervised Isolation Forest.
- Compute a composite operational risk score with four severity levels (LOW, MEDIUM, HIGH, CRITICAL).
- Manage complete incident lifecycles (Normal → Warning → Critical → Resolved) with operator acknowledgement notes and audit trails.
- Provide actionable root-cause mitigation advisories such as RET antenna calibration, PRB quota reallocation, slice QoS throttling and backhaul rerouting.
- Evaluate rigorously on a strictly chronological held-out test set, with no target leakage, and compare against a static-threshold baseline.
- Deliver an operator-facing dashboard, a documented REST API and a reproducible one-command deployment.

3. Design & Methodology

3.1 System Architecture

The system is organised as a pipeline. The simulator produces multi-cell telemetry, feature engineering turns raw KPIs into model inputs, the supervised and unsupervised models score each observation, and the risk scorer and alert manager convert scores into severity levels and incidents. A FastAPI service exposes all of this to the React operations centre, and also publishes metrics to Prometheus and events to a mock ONAP VES dispatcher.

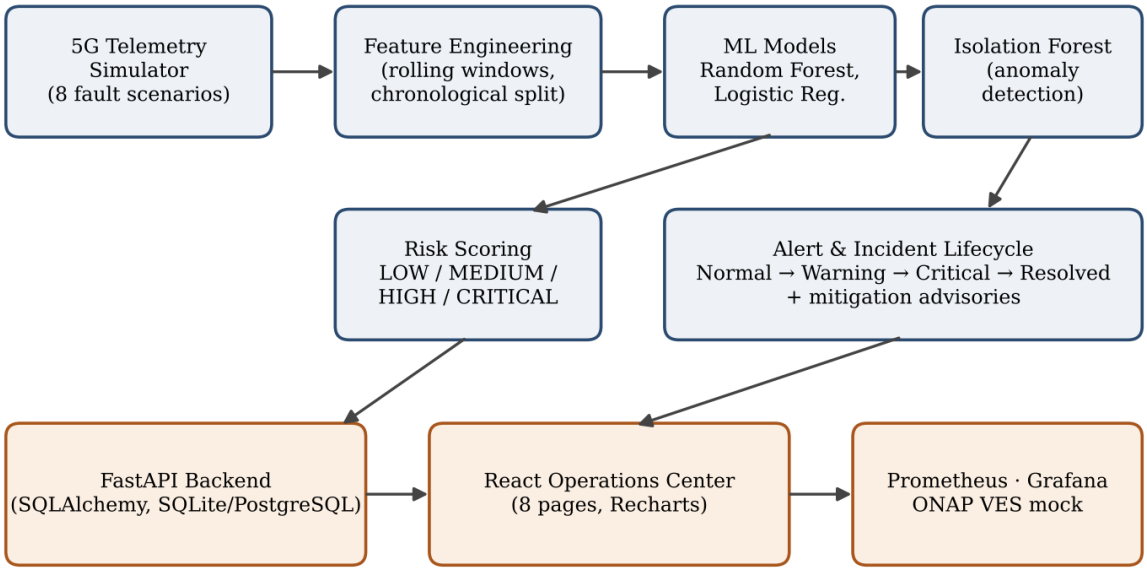


Figure 3.1: High-level architecture of the fault prediction platform

3.2 Technology Stack

Table 3.1: Technology stack

Layer	Technologies
Backend	Python 3.11+, FastAPI, Uvicorn, Pydantic, SQLAlchemy, SQLite (default) / PostgreSQL (cloud-ready), Prometheus Client
Machine learning	Scikit-Learn, pandas, NumPy, Joblib
Frontend	React 18, Vite, TypeScript, Tailwind CSS, Lucide Icons, Recharts
Monitoring & integration	Prometheus /metrics endpoint, Grafana provisioned dashboards, ONAP VES 7.2 mock event dispatcher
Deployment	Vercel (frontend), Render (backend), Docker Compose (local / on-premises)

3.3 Machine Learning Methodology

- **Supervised models.** A Random Forest classifier is the primary predictor and a Logistic Regression model is the interpretable baseline. Both predict whether a fault will occur within the forecast horizon.

- **Unsupervised model.** An Isolation Forest scores how unusual a multivariate observation is, allowing detection of zero-day anomalies that match none of the known fault signatures.
- **Leakage-free evaluation.** Models are trained with a strict chronological split, so the test set contains only data that occurs after the training period. This avoids target leakage and reflects real deployment, where the future is never available at training time.
- **Persistence.** Trained models are stored with Joblib in artifacts/models/ and backend/app/ml/saved_models/, so they load in under 50 ms and the service needs no retraining at startup.
- **Baseline.** Static 3GPP-style thresholds are evaluated on the same test set to quantify the benefit of the learned models.

3.4 Supported Fault Scenarios

The simulator can inject eight fault scenarios, each representing a distinct failure mode of the radio, transport or core network.

Table 3.2: The eight supported 5G fault scenarios

Scenario	Signature
radio_degradation	RSRP drops to −130 dBm and SINR to −4 dB (antenna miscalibration, feeder loss)
cell_congestion	PRB utilisation spikes to 98% and active UEs surge past 500 (stadium, flash crowds)
packet_loss_instability	Fronthaul/eCPRI frame drops of 15–30% with jitter
increased_latency	Round-trip latency above 250 ms, breaching 5G URLLC bounds
throughput_degradation	User-plane throughput below 15 Mbps under nominal load
cpu_memory_saturation	Containerised NFVI CPU/memory at 98%, causing resource starvation
backhaul_interruption	Complete transport cut with 85% loss and link flapping
service_availability_degradation	Core availability below 45%, AMF registration failures

3.5 Risk Scoring and Incident Lifecycle

The prediction probability and the anomaly score are combined into a composite risk score that is mapped to LOW, MEDIUM, HIGH or CRITICAL. Each incident then moves through the lifecycle Normal → Warning → Critical → Resolved. Operators can acknowledge an alert with a note and resolve it through a dialog, producing an audit trail. For each fault type the system attaches a mitigation advisory, for example RET antenna calibration for radio degradation, PRB quota reallocation for congestion, slice QoS throttling for overload, and backhaul rerouting for transport failures.

4. Implementation & Functionality

4.1 Backend

The backend is a FastAPI application served by Uvicorn (backend.app.main:app). Pydantic models validate requests and responses, SQLAlchemy persists telemetry, alerts and audit records to SQLite by default or PostgreSQL in the cloud, and the Prometheus client publishes operational metrics at /metrics. Interactive Swagger documentation is generated automatically at /docs.

4.2 Operations Center Frontend

The frontend is a single-page React 18 application built with Vite and TypeScript, styled with Tailwind CSS in a dark NOC aesthetic, with Recharts for live charts. It provides eight pages:

Table 4.1: Frontend pages

Page	Route	Function
Executive Overview		Real-time KPIs, active alert count, live sparklines, quick fault injection
Network Monitoring	monitoring	Multi-site and cell filters; four synchronised live charts (radio quality, transport, throughput, compute)
AI Fault Prediction	prediction	Live probability gauge, predicted lead time, top feature breakdown, ad-hoc test runner
Anomaly Detection	anomalies	Isolation Forest score timeline, decision boundary, anomaly table
Alert Center	alerts	Status and severity filters, search, acknowledge with notes, resolve modal
Model Evaluation	evaluation	Test-set benchmarks, confusion matrix, threshold comparison chart, feature rankings
5G Simulator Controls	simulator	Eight fault scenario trigger cards, target selectors, live scenario countdown
Settings & System Health	health	API connectivity, database stats, Prometheus exporter status, OpenAPI endpoint catalogue

4.3 Monitoring and Integration

Prometheus scrapes the backend /metrics endpoint and Grafana is pre-provisioned with dashboards. A mock event dispatcher emits ONAP VES 7.2 formatted events, demonstrating how alerts could be passed to an ONAP-based automation layer.

4.4 Running the System Locally

The backend and frontend run in two terminals. Pre-trained model weights are already included in the repository, so retraining is optional.

Terminal 1 – Backend:

```
pip install -r requirements.txt
python scripts/train_and_eval.py      # optional: retrain / verify metrics
uvicorn backend.app.main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 2 – Frontend:

```
cd frontend
npm install
npm run dev
```

The API is then available at http://localhost:8000 (Swagger at /docs, metrics at /metrics) and the operations centre at http://localhost:5173.

4.5 Docker Deployment

The full stack (FastAPI, React, Prometheus and Grafana) is launched with a single command:

```
docker-compose up --build -d
```

Table 4.2: Docker Compose service endpoints

Service	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000
Prometheus	http://localhost:9090
Grafana	http://localhost:3001 (admin / admin)

For cloud deployment, the backend targets Render and the frontend targets Vercel; setup instructions are in docs/setup.md, with architecture and methodology notes in docs/architecture.md and docs/methodology.md.

4.6 Testing

An automated suite covers the simulator, feature engineering, models, alert logic, API, risk scoring and an end-to-end smoke test. It is run with python -m pytest tests -v. All 16 tests passed in 30.26 seconds (100% pass rate).

5. Result & Discussion

5.1 Evaluation Setup

All models were evaluated on a held-out temporal test set of 3,150 samples, drawn strictly after the training period. The Random Forest is the primary model, Logistic Regression is the learned baseline, and static 3GPP-style thresholds represent current reactive practice.

5.2 Quantitative Results

Table 5.1: Model comparison on the held-out temporal test set

Metric	Random Forest (primary)	Logistic Regression (baseline)	Static 3GPP thresholds
Precision	99.32%	100.00%	100.00%
Recall (sensitivity)	100.00%	99.31%	60.00%
F1-score	99.66%	99.65%	75.00%
ROC-AUC	1.0000	1.0000	0.7000
PR-AUC	1.0000	0.9990	0.6500
False positive rate	0.03%	0.00%	0.00%
Missed fault rate (FNR)	0.00%	0.69%	40.00%
Inference latency	0.0431 ms/pt	0.0120 ms/pt	<0.001 ms/pt
Lead-time horizon	15–22.5 min	15–22.5 min	0 min (reactive)

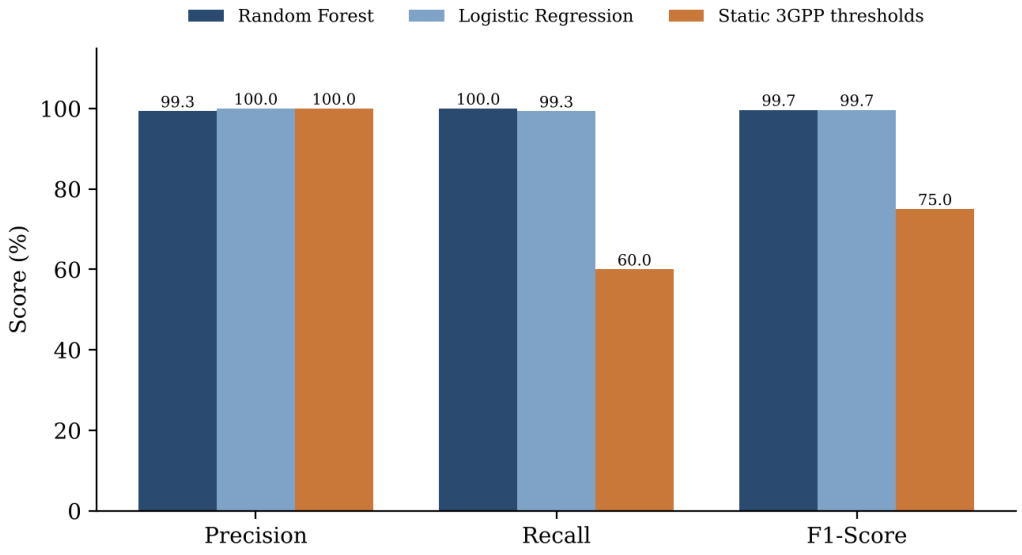


Figure 5.1: Precision, recall and F1-score by approach

5.3 Confusion Matrix

On the 3,150-sample test set the Random Forest produced 3,004 true negatives, 1 false positive, 0 false negatives and 145 true positives.

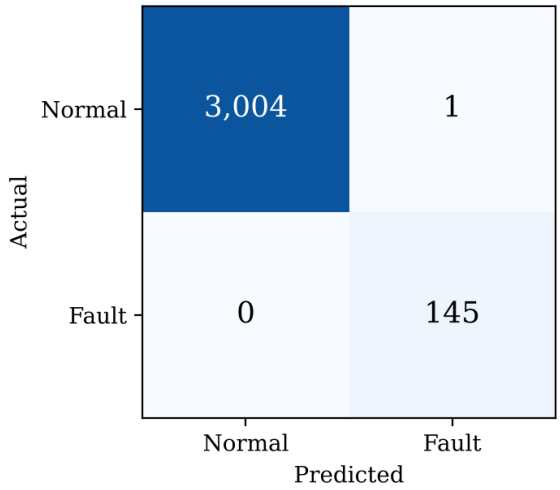


Figure 5.2: Random Forest confusion matrix on the test set

5.4 Discussion

- **No missed faults.** The Random Forest achieved 100% recall with a missed-fault rate of 0%, whereas static thresholds missed 40% of faults. In operations, a missed fault is costlier than a false alarm, so this is the most important result.
- **Very low false-alarm rate.** Only 1 of 3,005 normal samples was flagged (FPR 0.03%), so alert fatigue is not a concern on this data.
- **Early warning.** Both learned models give a 15–22.5 minute lead time, while threshold alarms give none. This window is the practical value of the system.
- **Speed.** Inference takes 0.0431 ms per point, far below any real-time telemetry interval, and models load in under 50 ms.
- **Logistic Regression is a strong baseline.** It reaches almost identical F1 (99.65%) with faster inference, which suggests the simulated fault signatures are largely separable; the Random Forest is preferred because it avoids any missed fault.

5.5 Limitations

The near-perfect scores should be read with care. The data is produced by a simulator whose fault signatures are well separated from normal behaviour, which makes classification easier than on real operator traces, and the test set contains only 145 fault samples. Results on live network data would likely be lower and should be validated on real or more diverse telemetry before operational use.

6. Conclusion

This project delivers an end-to-end, AI-driven fault prediction platform for 5G testbeds. By learning from multi-cell telemetry, it forecasts disruptions 15 to 22.5 minutes ahead, detects unseen anomalies with an Isolation Forest, scores operational risk, and guides operators through the incident lifecycle with concrete mitigation advice. On a leakage-free chronological test set, the Random Forest reached 99.32% precision and 100% recall (F1 99.66%) with no missed faults, compared with 60% recall for static thresholds. The system is packaged as a FastAPI backend and a React operations centre with Prometheus/Grafana monitoring, is covered by 16 passing automated tests, and can be deployed locally, with Docker Compose, or on Render and Vercel.

6.1 Future Work

- Validate and fine-tune the models on real operator or open O-RAN telemetry to confirm generalisation beyond simulated data.
- Add online learning and drift detection so models adapt as traffic patterns change.
- Replace the mock ONAP VES dispatcher with a live closed-loop integration that can trigger mitigation automatically.
- Extend root-cause analysis with feature attribution (for example SHAP) for each prediction.